



Tribhuvan University
Faculty of Humanities and Social Science
Himalaya College Of Engineering

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project is prepared under my supervision by Binit Maharjan entitled “**Traffic Violation Report System**” in the partial fulfillment for the degree of Bachelors of Computer Application for the final

Signature

Er. Himel Chand Thapa

Supervisor

Humanities and Social Science

Himalaya College of Engineering

Chyasal-9, Lalitpur



Tribhuvan University
Faculty of Humanities and Social Science
Himalaya College Of Engineering

LETTER OF APPROVAL

This is to certify that this project report prepared by Shashank Singh on “Traffic Violation Report System” in partial fulfilment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required

Er. Himal Chand Thapa Supervisor Department of Computer application Himalaya College of Engineering Chyasal-9, Lalitpur	Er. Himal Chand Thapa Co-Ordinator Department of Computer application Himalaya College of Engineering Chyasal-9, Lalitpur
Signature of Internal Examiner	Signature of External Examiner

ACKNOWLEDGEMENT

This project on developing a Traffic Violation Reporting System, which enables users to report traffic violations with photographic evidence and location data, while providing features such as license plate recognition, violation hotspot detection, and centralized report management, could not have been achieved without the kind support of many individuals.

I am highly thankful to **[Supervisor's Name]** Project Supervisor and Coordinator, for their kind guidance, valuable suggestions, and encouragement throughout the course of developing this project. Their knowledge of the subject, as well as their continuous support, helped me understand and implement various concepts related to web and mobile application development, database management, geospatial analysis, and OCR-based license plate recognition.

I would also like to express my gratitude towards all the individuals, including my teachers, mentors, friends, and classmates, who provided valuable suggestions, ideas, and feedback during the development and testing phases of the project. Their support helped improve the performance, functionality, and overall quality of the Traffic Violation Reporting System.

Finally, I would like to express my sincere thanks to all the individuals who contributed to the successful completion of this project, either directly or indirectly.

ABSTRACT

Traffic violations are a major challenge in urban areas of Nepal, where existing reporting and enforcement processes often depend on manual monitoring and informal reporting methods. The lack of a centralized system for recording, verifying, and analyzing traffic violations makes it difficult to efficiently manage reports, identify violation hotspots, and track repeat offenders. To address these challenges, this project presents a Traffic Violation Reporting System, a technology-driven platform that enables citizens to report traffic violations using photographic evidence and location information.

The system consists of a cross-platform mobile application for citizens, a web-based administrative dashboard for traffic authorities, and a backend service for managing reports, users, notifications, and data processing. The mobile application is developed using React Native and Expo, while the administrative dashboard uses React and Vite. The backend is implemented using Express, TypeScript, and MongoDB. A custom Optical Character Recognition (OCR) pipeline using TensorFlow/Keras and OpenCV is incorporated to assist in recognizing Nepali vehicle license plates from submitted images.

The system also incorporates geospatial and data-processing techniques to provide additional analytical capabilities. The Haversine formula is used for distance-based filtering, while DBSCAN clustering is used to identify areas with a high concentration of reported violations. Levenshtein distance is applied to match OCR-extracted plate numbers with existing records despite minor recognition errors. Real-time communication and notifications are implemented using Socket.io and Firebase Cloud Messaging.

The developed system demonstrates how citizen participation, automated license plate recognition, geospatial analysis, and centralized administration can be integrated into a single platform for structured traffic violation reporting. Testing confirmed the functionality of the major system components, while limitations were primarily observed in OCR character segmentation for low-quality or unclear license plate images. The system therefore provides an assistive reporting and management platform that can be further enhanced through improved OCR models, integration with official vehicle registration systems, offline support, automated fine processing, and video-based violation detection.

TABLE OF CONTENTS

ACKNOWLEDGEMENT.....	III
ABSTRACT.....	IV
TABLE OF CONTENTS	V
LIST OF ABBREVIATIONS	VII
LIST OF FIGURES.....	VIII
LIST OF TABLES.....	IX
1 CHAPTER 1: INTRODUCTION.....	1
1.1 INTRODUCTION	1
1.2 PROBLEM STATEMENT	2
1.3 OBJECTIVES	2
1.4 SCOPE AND LIMITATION.....	2
1.4.1 Scope.....	2
1.4.2 Limitations	3
1.5 DEVELOPMENT METHODOLOGY	3
1.6 REPORT ORGANIZATION	5
2 CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW	6
2.1 BACKGROUND STUDY.....	6
2.2 LITERATURE REVIEW	7
3 CHAPTER 3: SYSTEM ANALYSIS AND DESIGN.....	9
3.1 SYSTEM ANALYSIS.....	9
3.1.1 Requirement Analysis	9
3.1.2 Feasibility Analysis.....	11
3.1.3 Object Modeling	15
3.1.4 Dynamic Modeling	15
3.1.5 Process Modelling using Activity Diagrams	18
3.2 SYSTEM DESIGN.....	20
3.2.1 Component Diagram.....	20

3.2.2	Deployment Diagram	21
3.2.3	ER Diagram	23
3.3	ALGORITHM DETAILS	23
3.3.1	Haversine Formula (Distance Calculation).....	23
3.3.2	DBSCAN (Density-Based Spatial Clustering of Applications with Noise) ..	24
3.3.3	Levenshtein Distance (Plate Matching)	25
4	CHAPTER 4: IMPLEMENTATION & TESTING	26
4.1	IMPLEMENTATION	26
4.1.1	Tools Used	26
4.2	TESTING	27
5	CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATION	32
5.1	CONCLUSION.....	32
5.2	FUTURE RECOMMENDATION	33
6	REFERENCES.....	35
7	APPENDICES.....	36
7.1	MOBILE APP	36
7.2	ADMIN	38

LIST OF ABBREVIATIONS

ALPR	Automatic License Plate Recognition
ANPR	Automatic Number Plate Recognition
API	Application Programming Interface
CNN	Convolutional Neural Network
CRNN	Convolutional Recurrent Neural Network
CTC	Connectionist Temporal Classification
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
ER	Entity-Relationship
FCM	Firestore Cloud Messaging
GPS	Global Positioning System
HOG	Histogram of Oriented Gradients
HTTPS	Hypertext Transfer Protocol Secure
iOS	iPhone Operating System
JPEG	Joint Photographic Experts Group
JWT	JSON Web Token
ML	Machine Learning
OCR	Optical Character Recognition
ODM	Object Data Modeling
OSM	OpenStreetMap
REST	Representational State Transfer
SDLC	Software Development Life Cycle
UI	User Interface
UX	User Experience
YOLO	You Only Look Once

LIST OF FIGURES

Figure 1.1: Agile Methodology.....	5
Figure 3.1: Use Case Diagram.....	9
Figure 3.2.1: Grantt Chart.....	14
Figure 3.3 State Diagram for Report Lifecycle	16
Figure 3.4 Sequence Diagram for Report Submission.....	17
Figure 3.5 Activity Diagram for Traffic Violation Reporting System	19
Figure 3.6 Component Diagram for Traffic Violation Reporting System.....	21
Figure 3.7Deployment Diagram	22
Figure 3.8: ER Diagram.....	23
Figure 7.1 Home Screen	36
Figure 7.2 Reports Screen.....	36
Figure 7.3 Report Modal.....	37
Figure 7.4New Report.....	37
Figure 7.5 Hotspots Screen	38
Figure 7.6 Admin Dashboard.....	38
Figure 7.7 Reports.....	39
Figure 7.8 Flagged Plates.....	39
Figure 7.9 Hotspots Map.....	40

LIST OF TABLES

Table 4.1: Test Case for Authentication Model	27
Table 4.2: Report Submission Module	28
Table 4.3: OCR & Plate Recognition Module	29
Table 4.4: Algorithm Module	30

CHAPTER 1: INTRODUCTION

1.1 Introduction

Road traffic is one of the most critical infrastructural systems in any country, and its safe and orderly management directly affects public safety, economic productivity, and quality of urban life. In Nepal, rapid urbanization and a sharp rise in the number of registered vehicles over the past decade have placed increasing pressure on traffic management systems that were largely designed for manual, in-person enforcement. Traffic police personnel are often required to physically monitor intersections, issue tickets on the spot, and manually record violation details a process that is labour intensive, inconsistent, and difficult to scale across a growing number of vehicles and road users.

At the same time, the widespread adoption of smartphones and mobile internet in Nepal presents an opportunity to involve citizens directly in the traffic enforcement process. Crowdsourced reporting where ordinary citizens document and report violations they witness has proven effective in many countries as a supplementary enforcement mechanism, extending the reach of traffic authorities beyond what physical patrolling alone can achieve. However, for such a system to be effective, it must be supported by an efficient backend infrastructure capable of verifying reports, identifying vehicles, and turning raw data into actionable enforcement intelligence.

Existing traffic violation reporting practices in Nepal remain largely informal, often relying on photographs shared over social media or word-of-mouth complaints, with no structured system to track outcomes or hold violators accountable. This creates a gap between citizen awareness and effective enforcement action. Additionally, even when reports are made, identifying the exact vehicle from an image requires manual reading of license plates a slow process, especially when Nepali plates use Devanagari script, which is not well-supported by conventional off-the-shelf OCR tools trained primarily on Latin-script plates.

By combining citizen participation, automated recognition, and data-driven analytics, the system aims to modernize how traffic violations are reported, verified, and acted upon in the Nepali context while remaining lightweight enough to be practically deployable using freely available mapping and cloud services.

1.2 Problem Statement

The Traffic Violation Reporter tries to solve the following problems:

- Traffic violations often go unreported due to lack of an easy, accessible reporting channel for the public.
- Traffic authorities lack a centralized system to track, analyse, and act on violation trends across different locations.
- Manual identification of vehicle license plates from reported images is time-consuming and error-prone.
- There is no effective mechanism to identify violation hotspots or repeat offenders (flagged vehicles) using existing data.

1.3 Objectives

The main objective of this project are as follows:

- To develop a cross platform mobile application that allows citizens to report traffic violations with photo evidence and location data.
- To implement an automated Nepali license plate recognition (OCR) pipeline to assist in identifying vehicles from violation images.
- To provide data-driven insights such as violation hotspot mapping and analytics for informed traffic enforcement decisions.

1.4 Scope and Limitation

1.4.1 Scope

The scopes of this project include:

- Mobile Application for citizens to submit violation reports with images, location, and details.
- Admin Dashboard for authorities to manage reports, view analytics, and track hotspots on an interactive map.

- Backend & OCR Pipeline for handling data storage, business logic, real-time communication, and automated license plate recognition.

1.4.2 Limitations

Limitations of the project are listed below:

- The Nepali license plate OCR pipeline, being a custom-trained model rather than a commercially matured solution, may not achieve high accuracy under poor lighting, motion blur, unusual camera angles, damaged/dirty plates, or non-standard plate formats. Character segmentation errors can still occur, especially with visually similar Devanagari characters.
- The system does not connect to Nepal's official vehicle registration database (e.g., Department of Transport Management records), so it cannot automatically verify vehicle ownership details or cross-check registration status it only stores and flags plate numbers as reported.

1.5 Development Methodology

This project was developed using the Agile Software Development Methodology. Agile was chosen over traditional models such as the Waterfall model because the system involves multiple interdependent yet independently testable components a mobile application, an admin dashboard, a backend API, and a machine learning based OCR pipeline each of which required experimentation, testing, and refinement rather than a single fixed upfront design. Given that requirements, particularly around OCR accuracy and system architecture, evolved as development progressed, an iterative approach allowed the design to adapt based on results and feedback obtained at each stage, rather than committing to a rigid plan that could not accommodate changes discovered mid-development.

The project began with identifying the core problem the lack of an efficient, technology-driven channel for reporting and managing traffic violations in Nepal and translating this into concrete functional requirements. These included citizen-side reporting with photo and location capture, an authority-side dashboard for verification and analytics, and an automated mechanism to identify vehicles from submitted images. Non-functional requirements such as real-time communication, usability, and reliance on freely available

services (e.g., OpenStreetMap instead of paid mapping APIs) were also considered at this stage.

Based on the gathered requirements, the overall system architecture was designed as a three-tier structure: a React Native (Expo) mobile client, a React admin dashboard, and an Express/TypeScript backend with MongoDB as the data store. Core algorithms including Haversine distance calculation, DBSCAN clustering for hotspot detection.

Rather than building the entire system in one pass, development proceeded in incremental cycles, with each cycle targeting a specific, testable feature. Early iterations focused on establishing the reporting flow allowing a user to submit a violation report with an image, description, and location. Subsequent iterations added real-time capabilities using Socket.io and Firebase Cloud Messaging, followed by the admin-side mapping and analytics features using Leaflet and Recharts. The Nepali license plate OCR pipeline, being the most experimental component, was developed as a separate iterative track it was initially integrated as a microservice, removed after accuracy issues were identified, and later redeveloped as a more carefully evaluated standalone pipeline using TensorFlow/Keras and OpenCV.

Each module was tested independently as it was completed, allowing issues to be caught and resolved early rather than accumulating until final integration.

Throughout the development process, GitHub was used for version control, allowing incremental commits, tracking of changes across the mobile, dashboard, and backend codebases, and safe rollback in case a change introduced instability. This supported the iterative nature of the methodology by making it easy to isolate, test, and merge individual features without disrupting the overall system.



Figure 1.1: Agile Methodology

1.6 Report Organization

This report is organized into several chapters that describe the development of the Traffic Violation Reporting System in a systematic manner.

Chapter 1: Introduction presents the background of the project, problem statement, objectives, scope, significance, and methodology used for developing the system. It provides an overview of the existing challenges in traffic violation reporting and the proposed solution.

Chapter 2: Literature Review discusses existing systems, relevant technologies, and research related to traffic violation reporting, Automatic License Plate Recognition (ALPR), Optical Character Recognition (OCR), and geospatial analysis. It also identifies the research gap addressed by the proposed system.

Chapter 3: System Analysis describes the functional and non-functional requirements of the system. It also covers feasibility analysis and the technologies required for implementing the proposed system.

Chapter 4: System Design presents the overall architecture and design of the system. It includes object modeling, component architecture, database design, system workflow, and the interaction between the mobile application, admin dashboard, backend, and other system components.

Chapter 5: Implementation explains the implementation of the Traffic Violation Reporting System. It describes the development of the mobile application, admin

dashboard, backend services, database, license plate recognition system, geospatial features, real-time communication, and notification mechanisms.

Chapter 6: Testing presents the testing process and results of the developed system. It includes functional testing and other tests performed to verify the reliability and functionality of the major system components.

Chapter 7: Conclusion and Future Recommendations summarizes the work completed, major findings, and overall outcome of the project. It also presents possible future improvements, such as improved OCR accuracy, government database integration, automated fine management, offline support, and video-based traffic violation detection.

Chapter 2: Background Study and Literature Review

2.1 Background Study

Nepal has witnessed a rapid increase in vehicle registration over the past decade, particularly in urban centres such as Kathmandu Valley, driven by rising incomes, urban migration, and limited public transportation coverage. This growth in vehicle numbers has not been matched by a proportional expansion in traffic infrastructure or enforcement capacity. Traffic police in Nepal continue to rely heavily on manual patrolling, physical presence at intersections, and handwritten or basic digital ticketing systems. As a result, a large number of violations such as illegal parking, lane violations, signal jumping, and driving without required documentation go unnoticed or unreported simply because no officer is present at the time and location of the incident.

In the absence of a formal digital reporting channel, citizens who witness traffic violations in Nepal typically resort to informal methods: posting photos or videos on social media platforms (Facebook, Twitter/X, TikTok) tagging traffic police pages, or sharing them in community groups. While this raises public awareness, it lacks structure there is no standardized way to record the violation type, location, time, or vehicle identity, and there is no guaranteed mechanism ensuring that traffic authorities act on the report. This gap between citizen observation and administrative action formed a central motivation for studying and designing a structured reporting system.

Automatic License Plate Recognition (ALPR) systems have been widely researched and deployed internationally, generally relying on a pipeline of plate localization, character segmentation, and character recognition using techniques ranging from traditional image processing (edge detection, morphological operations) to deep learning-based object detection and OCR models. However, most publicly available ALPR models and datasets are trained on Latin-script plates common in Western countries. Nepali license plates use Devanagari script and follow region-specific formatting conventions, which are poorly supported by off-the-shelf OCR tools. This gap motivated the need for a custom-trained OCR pipeline, built using TensorFlow/Keras for model training and OpenCV for image preprocessing, tailored specifically to Nepali plate characteristics.

To move beyond simple report logging and provide actionable insight to traffic authorities, the study examined spatial data analysis techniques commonly used in crime mapping and urban analytics. The Haversine formula was studied as a method for calculating distances between geographic coordinates on the Earth's surface, useful for proximity-based filtering of reports. DBSCAN (Density-Based Spatial Clustering of Applications with Noise) was studied as a clustering algorithm well-suited to identifying geographic "hotspots" areas with a high density of reported violations without requiring a predefined number of clusters. These techniques informed the system's approach to hotspot detection and visualization on the admin dashboard.

2.2 Literature Review

One of the earliest works on Nepali plate recognition used Support Vector Machines with HOG features to classify characters, following a basic pipeline of plate detection, segmentation, and recognition [1]. This approach achieved only around 75% accuracy, showing the limits of older machine learning methods for this task.

A more recent study by Khadka et al. used YOLO to detect the plate and characters, then a CNN to classify 34 Devanagari characters, improving accuracy through data augmentation and extra training on embossed plates [2]. This deep learning-based approach is closer to the method used in this project's OCR pipeline.

Another study applied YOLOv8 along with object tracking to detect and follow vehicle plates across video frames [3], which is more suited to CCTV-style monitoring than the single-image reporting used in this project.

Research on Devanagari script OCR shows that character segmentation is harder than for Latin scripts due to touching/conjunct characters and visually similar letters [4]. This matches the character segmentation issues encountered during this project's OCR development.

A public dataset containing over 26,000 images across 34 Nepali plate characters is also available to support training and testing of OCR models [5], showing there are usable resources for this kind of work.

Broader studies on license plate recognition systems point out common problems such as poor lighting, bad camera angles, and worn plates affecting accuracy [6], reinforcing why this project still requires manual admin verification alongside automated OCR.

Research Gap: Most existing Nepali ANPR studies focus only on plate/character recognition accuracy in isolation. None combine OCR with real-time citizen reporting, hotspot detection, and repeat-offender tracking in a single deployable system — which is the gap this project addresses.

Chapter 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

The system analysis of the system is done by conducting requirement analysis, feasibility analysis, data modeling and process modelling as follows:

3.1.1 Requirement Analysis

3.1.1.1 Functional Requirement

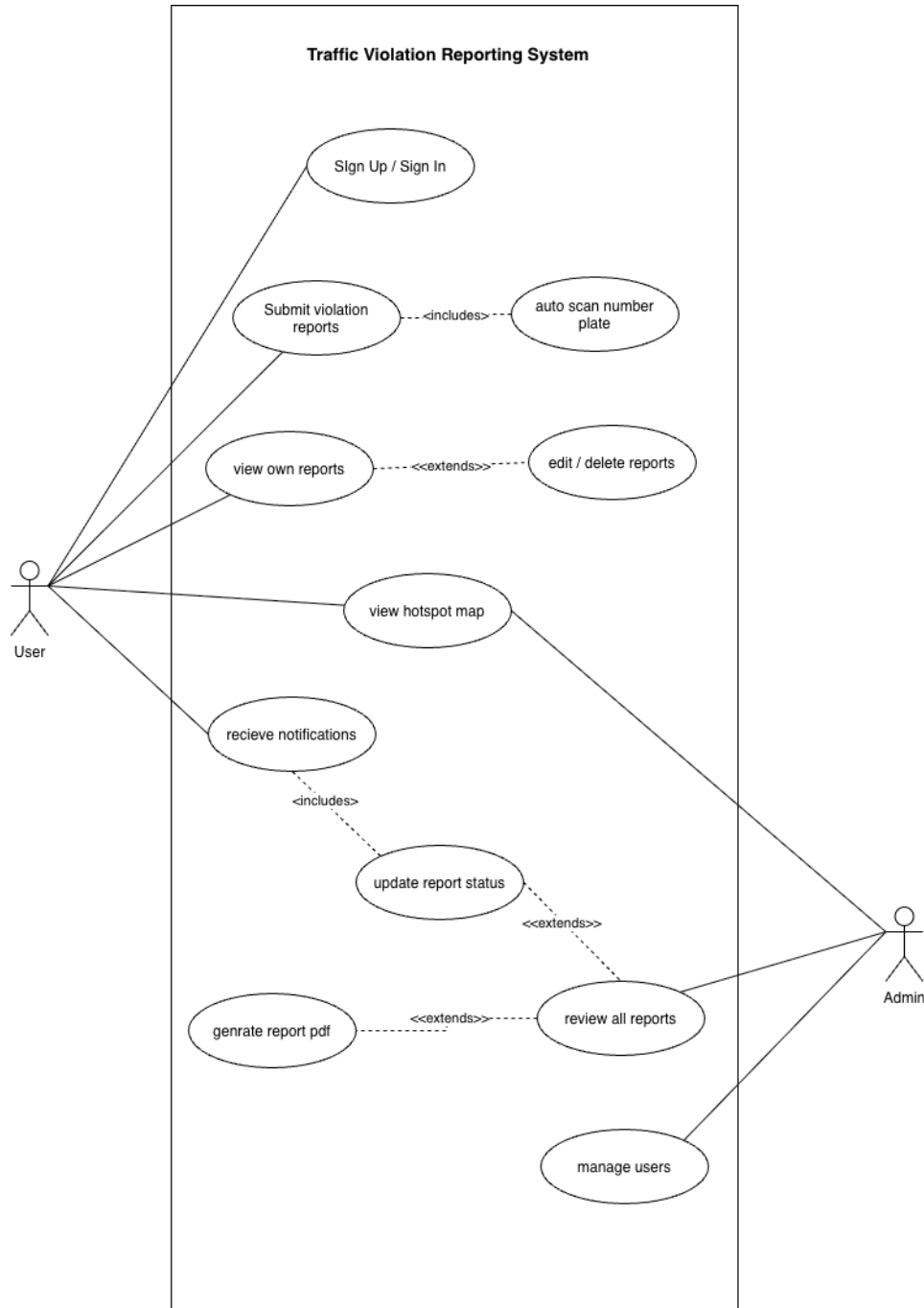


Figure 3.1: Use Case Diagram

- **Sign Up / Sign In** Users create an account or log in to access the system. Authentication is required before any other action can be performed.
- **Submit Violation Report:** Users capture or upload an image of a traffic violation, fill in details like number plate, violation type, description and pin the location on a map. The report is then sent to the server for admin review.
- **Auto Scan Number Plate:** Optionally extends submit report — the camera automatically detects and reads the number plate using OCR, pre-filling the number plate field so the user doesn't have to type it manually.
- **View Own Reports:** Users can see a list of all violation reports they have previously submitted, along with their current status (pending, approved, rejected).
- **Edit / Delete Report:** Optionally extends view own reports — after viewing a report the user can choose to edit its details or delete it entirely if it was submitted incorrectly.
- **View Hotspot Map:** Both users and admin can view an interactive map showing violation hotspots across Nepal, clustered using the DBSCAN algorithm to highlight high-violation areas.
- **Receive Notifications:** Users receive push notifications and in-app alerts when an admin updates the status of their submitted report.
- **Update Report Status:** Receive Notifications Admin can change a report's status to approved or rejected. This automatically triggers a notification to the user who submitted the report.
- **Review All Reports:** Update Report Status Admin can view all reports submitted by all users with filtering by status, violation type and distance. From here the admin can extend into updating the status of any report.

- **Generate Report PDF:** Extends review all reports — admin can generate and download a formatted PDF document for any individual report for official record keeping or printing.
- **Manage Users:** Admin can view all registered users, see how many reports each user has submitted and manage user accounts including roles and access.

3.1.1.2 Non-Functional Requirements

- **Scalability:** The platform must support a growing number of users, listings, and transactions without performance degradation.
- **Security:** Robust security measures to protect user data, including encryption, secure authentication, and regular security audits.
- **Performance:** The system should load quickly and handle high traffic during peak times, ensuring a smooth user experience.
- **Usability:** Intuitive and user-friendly interface designed for all user types, including those with limited technical skills.
- **Reliability:** The platform should ensure high availability with minimal downtime and provide backups for data protection.

3.1.2 Feasibility Analysis

3.1.2.1 Technical Feasibility

Technical feasibility is assessed by evaluating whether the proposed system can be built using currently available technologies, tools, and development skills. The Traffic Violation Reporting System is technically feasible given the maturity and free availability of all technologies used across its components. The mobile client is built using React Native with Expo, a well-established, well-documented framework for cross-platform Android/iOS development from a single codebase, while the backend uses Express with TypeScript and

MongoDB, a proven combination for building scalable APIs with flexible, semi-structured data storage suited to varying report formats. Real-time communication between citizens and admins is handled using Socket.io, a mature library requiring no custom protocol work, and mapping/location features rely on OpenStreetMap and Nominatim, which are free and open-source, avoiding the cost and API-key limitations of commercial alternatives such as Google Maps; supporting algorithms such as the Haversine formula and DBSCAN clustering are well-documented and straightforward to implement. The Nepali license plate OCR pipeline uses TensorFlow/Keras and OpenCV, both established, well-supported tools for building and training custom image recognition models, and while OCR accuracy depends on training data quality and preprocessing, this is a manageable constraint rather than a blocker, especially with manual admin verification as a fallback. Since all tools used are free/open-source and run on standard developer hardware (Windows/Mac) without requiring specialized infrastructure, the system is technically feasible to design, build, and deploy within the scope, timeline, and resource constraints of an academic project.

3.1.2.2 Operational Feasibility

The Traffic Violation Reporting System is operationally feasible as it is designed to fit naturally into existing behaviour patterns of both citizens and traffic authorities in Nepal, requiring minimal change in how people already interact with technology. Citizens are already accustomed to using smartphones to capture photos and share them on social media when they witness traffic violations, so shifting this behaviour into a dedicated reporting app involves a low learning curve, especially with a simple, intuitive interface for submitting reports with photos, descriptions, and location. On the administrative side, traffic authorities benefit from a centralized dashboard that consolidates reports, flags repeat offenders, and highlights violation hotspots, replacing scattered, informal reporting channels with a structured system that is easier to act on and track over time. The system does not require citizens or administrators to possess any specialized technical knowledge, and real-time notifications keep users informed without needing to actively check the app. Operational success does, however, depend on factors outside the system's direct control, such as institutional willingness among traffic authorities to actively monitor and act on the dashboard, and sufficient citizen adoption to generate a meaningful volume of reports.

3.1.2.3 Economic Feasibility

The Traffic Violation Reporting System is economically feasible as it has been designed to minimize both development and operational costs by relying entirely on free and open-source technologies. The mobile app (React Native/Expo), backend (Express/TypeScript/MongoDB), and OCR pipeline (TensorFlow/Keras, OpenCV) are all built using open-source frameworks with no licensing fees, and mapping/geocoding is handled through OpenStreetMap and Nominatim rather than paid services such as Google Maps, avoiding recurring API costs. Development tools such as GitHub for version control and Firebase Cloud Messaging's free tier for push notifications further keep costs at zero during the academic development phase, with the only real resource investment being developer time and standard computing hardware already available. For a real-world deployment beyond the academic scope, minor recurring costs could arise from server hosting, database hosting, and scaling notification services as user numbers grow, but these are modest and can be phased in gradually as adoption increases. Given the low development cost and the potential long-term benefit of improved traffic enforcement efficiency and public safety, the system is economically justified and feasible to build within a zero/near-zero budget academic setting.

3.1.2.4 Schedule Feasibility

The project was completed within the semester timeframe by mapping the work to standard Software Development Life Cycle (SDLC) phases, allowing progress to be tracked systematically while still permitting iteration within each phase.

1. **Planning:** Defined the project scope, objectives, and feasibility (technical, operational, economic, schedule), and identified the overall problem the system aims to solve.
2. **Requirement Analysis:** Gathered functional and non-functional requirements for the mobile app, admin dashboard, backend, and OCR pipeline, based on the identified problem of inefficient manual traffic violation reporting.
3. **System Design:** Designed the overall three-tier architecture, database schema (MongoDB), API structure, and core algorithms (Haversine distance, DBSCAN).

clustering, priority scoring), along with UI/UX wireframes for the mobile app and dashboard.

4. **Implementation:** Developed the system iteratively across its components — backend APIs and database, the mobile app (React Native/Expo), the admin dashboard (React/Vite), the OCR pipeline (TensorFlow/Keras, OpenCV, including an initial microservice attempt that was later redeveloped as a standalone pipeline), and real-time features (Socket.io) with push notifications (Firebase FCM).
5. **Testing:** Tested modules independently as they were completed (e.g., OCR evaluation tooling for character segmentation accuracy, Android/Expo build debugging), followed by integration testing across the full system once components were combined.
6. **Deployment:** Finalized builds, packaged the mobile app, and deployed the backend/dashboard for demonstration purposes.
7. **Documentation and Maintenance Planning:** Compiled the final project report, prepared presentation materials, and documented known limitations and future enhancement areas for potential post-submission maintenance.

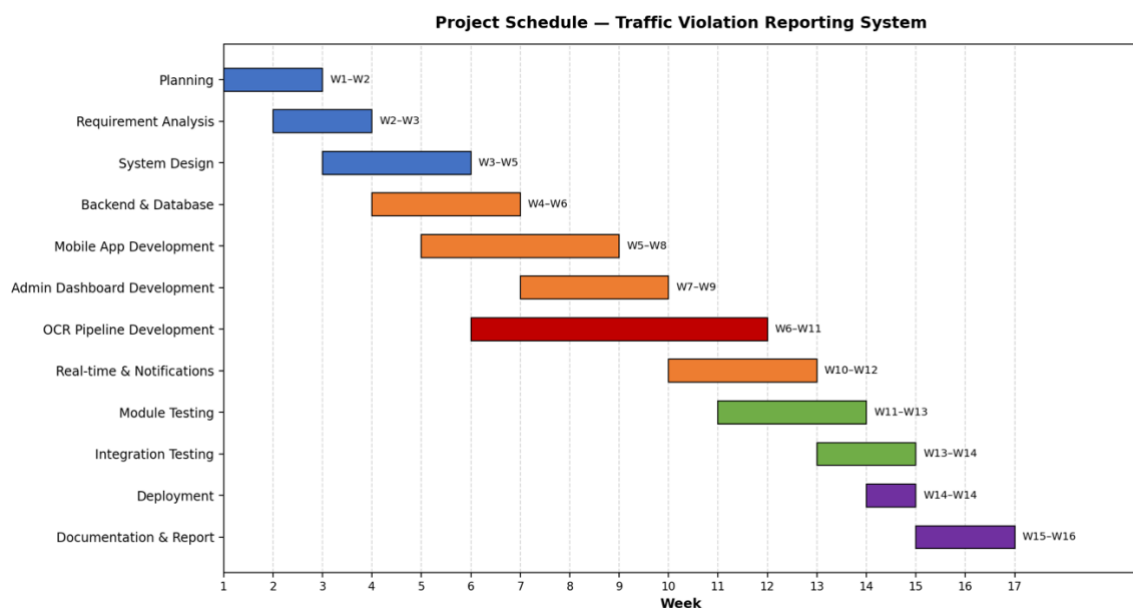


Figure 3.2.1: Gantt Chart

3.1.3 Object Modeling

The application is an efficient Traffic Violation Reporting System that is used by citizens to report traffic violations and by traffic authorities to verify, manage, and act on these reports. The design of the application consists of three entities, User, Report, and Notification, whereby a user may submit several reports, and each report may generate several notifications.

The User entity stores essential account information such as `_id`, name, email, password, phone, role, pushToken, and createdAt/updatedAt. This class is responsible for authentication, authorization, and distinguishing access levels between citizens and administrators, while pushToken enables delivery of real-time push notifications to the user's device. The Report entity represents the core functionality of the system. It stores violation details including number_plate, violation, description, image, location, and status. Each report is linked to a specific user through reportedBy, establishing a one-to-many relationship between User and Report. The location field embeds latitude, longitude, and a name for the reported site, supporting geospatial operations such as hotspot clustering and distance-based filtering. The status field tracks the report's lifecycle as it moves from submission through verification. The Notification entity manages status updates sent back to users, storing title, message, isRead, and references to both userId and reportId, establishing one-to-many relationships from User and Report to Notification. All reporting operations are managed through the Report module, which processes submitted evidence, applies automated license plate recognition, and stores structured results for admin review, while the Notification module ensures users remain informed of changes to their reports in real time. This design keeps the system modular, scalable, and easy to extend with additional detection algorithms or violation categories in the future.

3.1.4 Dynamic Modelling

Dynamic modelling captures the behavioural aspects of the Traffic Violation Reporting System — how objects interact over time and how a report changes state throughout its lifecycle. This is represented using a State Diagram, showing the lifecycle of a Report, and a Sequence Diagram, showing the interaction flow when a user submits a report.

3.1.4.1 State Diagram — Report Lifecycle

A Report object moves through a defined set of states from the moment it is submitted by a citizen to its final resolution by an administrator. It begins as Submitted, moves to Processing while the OCR pipeline attempts plate extraction, then to Pending once a plate number is attached. An administrator opening the report moves it to Under Review, after which it is either Verified or Rejected. A verified report whose plate matches a pattern of repeated violations is additionally marked Flagged, and once appropriate action is taken, both Verified and Flagged reports move to Closed. This ensures every report has a clear, auditable status at all times.

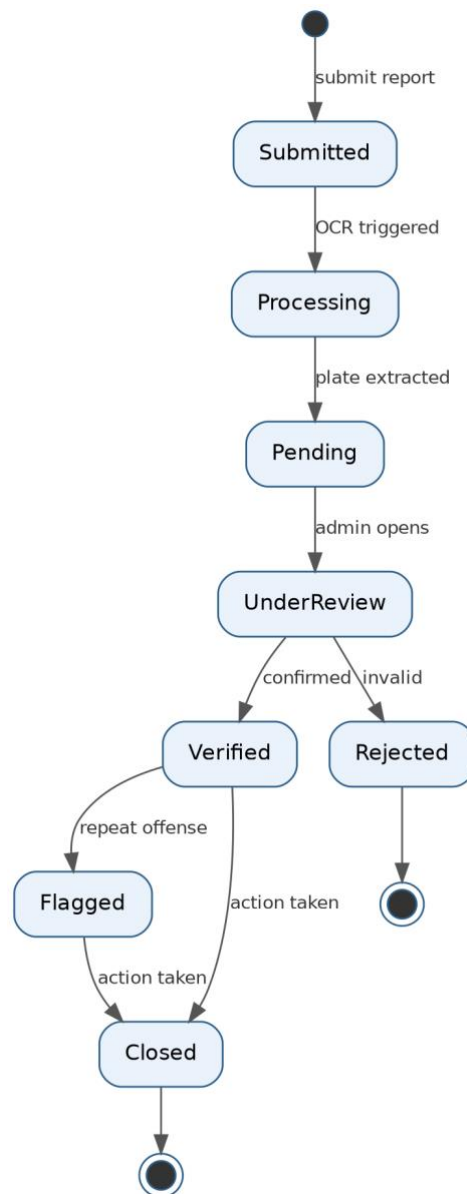


Figure 3.3 State Diagram for Report Lifecycle

3.1.4.2 Sequence Diagram — Report Submission Flow

The sequence diagram illustrates the interaction between the Mobile App, Backend Server, OCR Service, Database, and Admin Dashboard when a citizen submits a violation report. The user submits a report to the backend, which immediately saves it before sending the image to the OCR service for plate recognition. Once the plate number is returned, the backend updates the report, confirms submission to the user, and pushes the new report to the admin dashboard in real time via Socket.io. The administrator then reviews and verifies or rejects the report, after which the backend updates the report's status, creates a notification record, and sends a push notification back to the user.

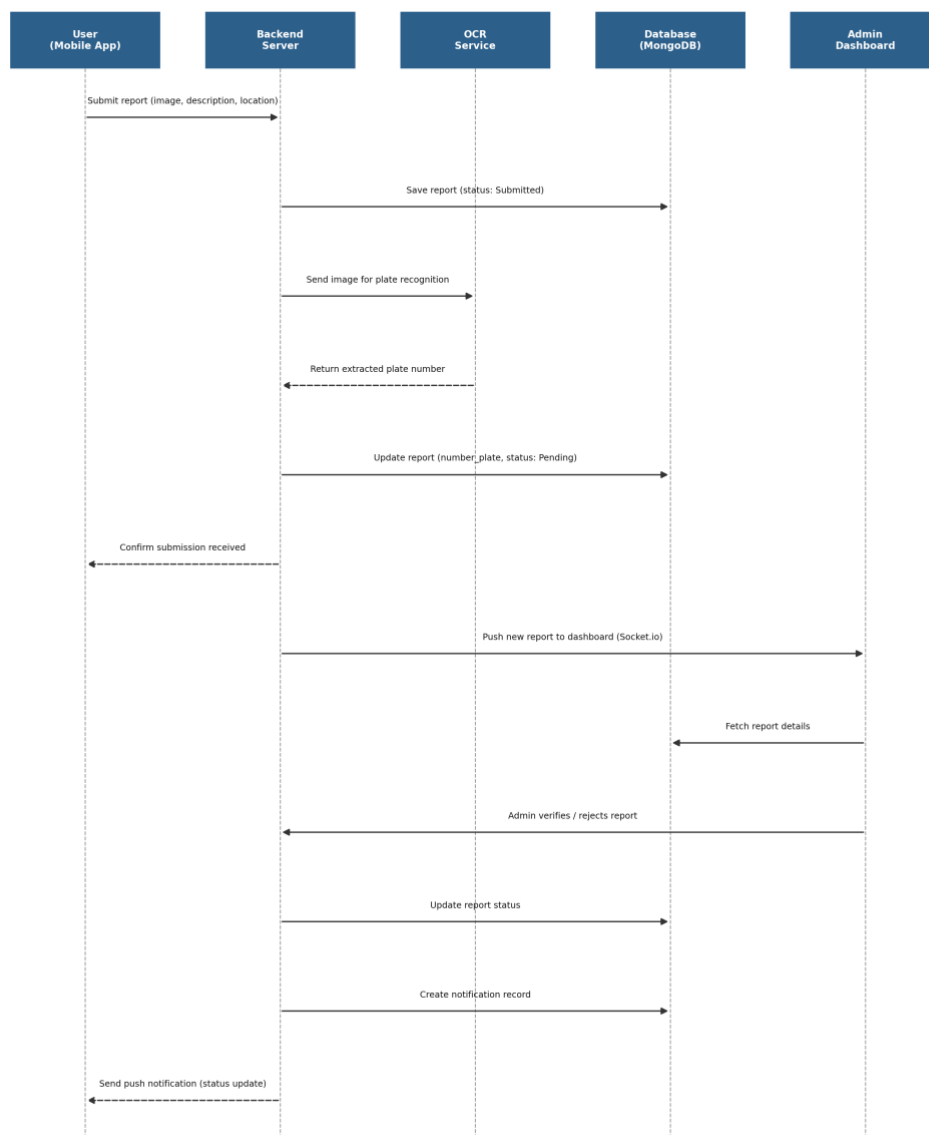


Figure 3.4 Sequence Diagram for Report Submission

3.1.5 Process Modelling using Activity Diagrams

Activity diagrams model the dynamic workflow of a process, capturing the sequence of actions, decision points, and parallel activities involved as a report moves from submission to resolution. The process begins when a user opens the mobile app and captures or selects a photo of the violation, along with a description and location. Upon submission, the system performs an automatic OCR check to attempt plate extraction; if this succeeds, the extracted plate number is attached to the report, and if it fails, the report is still saved and flagged for manual plate entry by an administrator. The report then enters the admin queue, where an administrator reviews the evidence and makes a decision — if the report is valid, it is marked verified and checked against prior records to determine whether the vehicle should be flagged as a repeat offender; if invalid, it is rejected and the process ends. Verified (and possibly flagged) reports proceed to an action-taken step, after which the report is closed and the citizen is notified of the outcome, completing the process for both the citizen and the administrator involved.

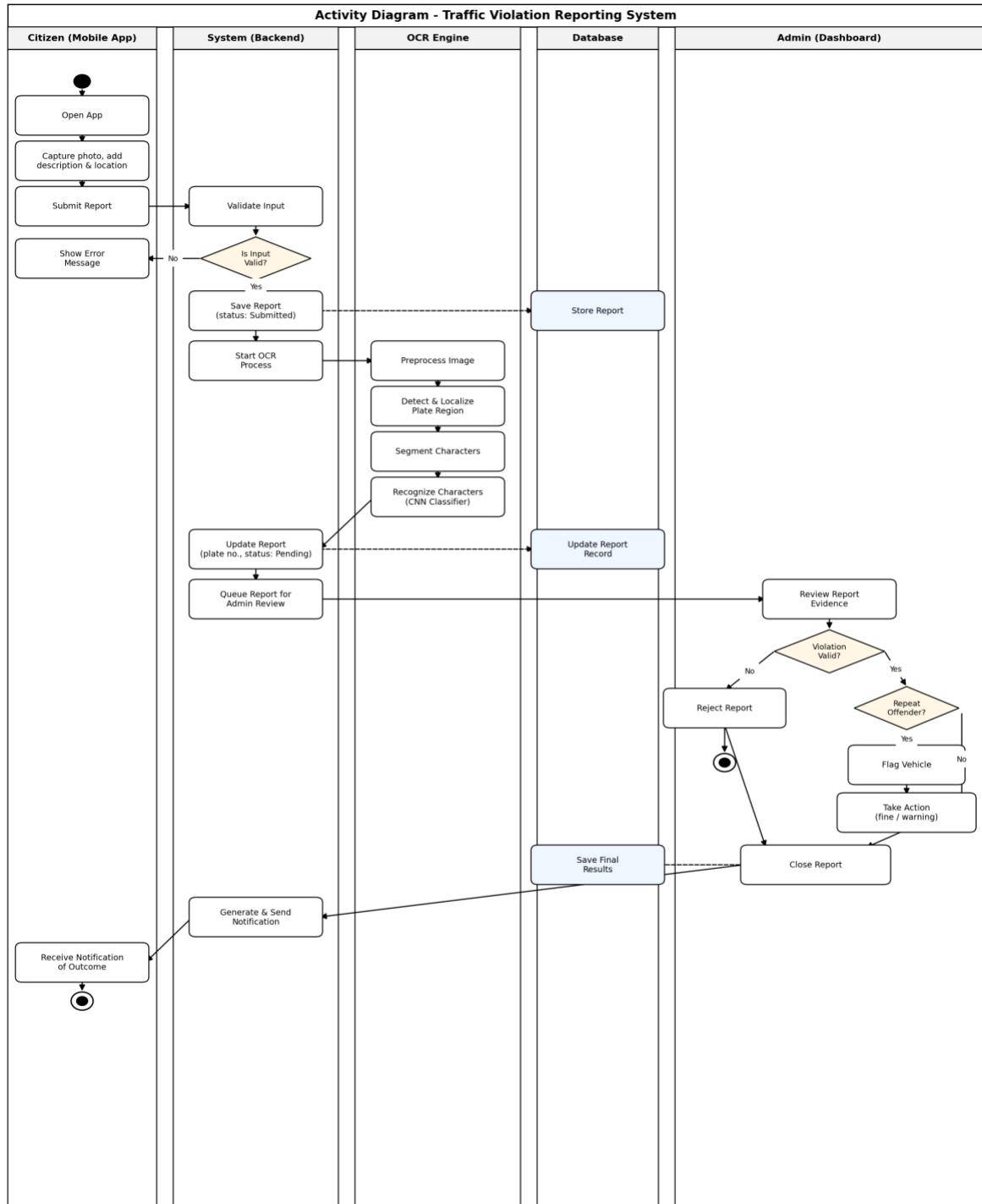


Figure 3.5 Activity Diagram for Traffic Violation Reporting System

3.2 System Design

3.2.1 Component Diagram

The component diagram provides a representation of the Traffic Violation Reporting System's architecture. The first element is the user's device, equipped with the Mobile Application (React Native/Expo) interface, through which citizens capture violation evidence, enter location details, and submit reports. Using the Mobile App Component, the user performs report submission, status tracking, and notification-viewing tasks. Input data is passed through the Backend Application Component, which interfaces with the OCR & Detection Engine. This engine consists of different modules such as the image preprocessing module, plate localization module, character segmentation module, and character recognition module, as well as supporting modules for hotspot clustering (DBSCAN), distance filtering (Haversine), and repeat-offender flagging. After extracting the plate number and analysing the report, the system stores all information into the Database Component, which holds user data, report records, and notification logs. The Admin Dashboard Component, running on the traffic authority's browser, retrieves report and analytics data from the backend for review, verification, and hotspot visualization. At the final stage, the Notification Component receives status updates from the backend and delivers real-time alerts to citizens via Firebase Cloud Messaging.

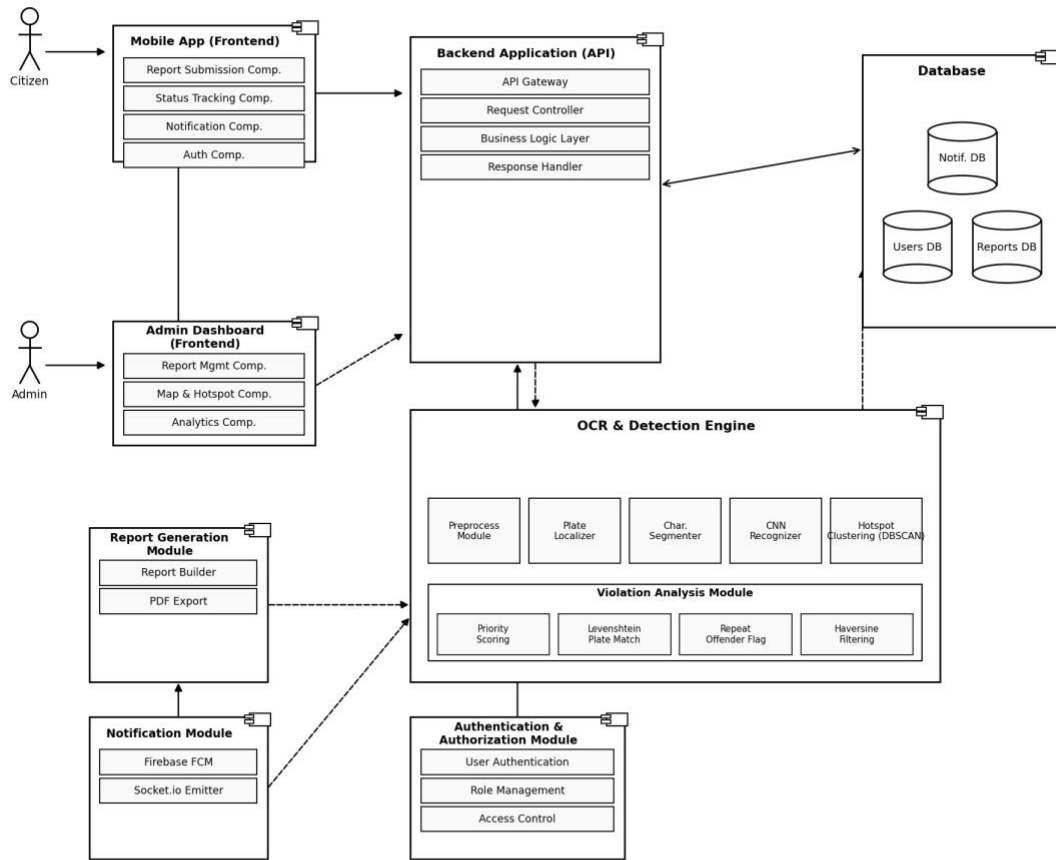


Figure 3.6 Component Diagram for Traffic Violation Reporting System

3.2.2 Deployment Diagram

The deployment diagram of the Traffic Violation Reporting System shows how the user interacts with the components that make up the system's infrastructure while using the system. First, the citizen accesses the system via the Mobile App and submits a report — including a photo, description, and location — as an HTTPS request. The data collected from the user is sent to the Application Server, which manages user requests and authenticates users. Then, the application server interacts with the OCR & Detection Engine, where components such as the plate localizer, character segmenter, and CNN recognizer run to extract the vehicle's license plate number. Meanwhile, the application server also communicates with the Admin Dashboard, which traffic authorities access via

a web browser to review, verify, and manage reports. Data on reports, users, and notifications is stored in the Database Server, and uploaded violation images are stored in a separate File Storage system. Push notifications are delivered to users through the Firebase Cloud Messaging Server. After a report is reviewed and processed, the updated status is sent back to the Mobile App, where it is displayed to the user in real time. The diagram highlights the interaction between the mobile client, application server, OCR engine, database server, file storage, and the admin dashboard to perform automated violation reporting and management.

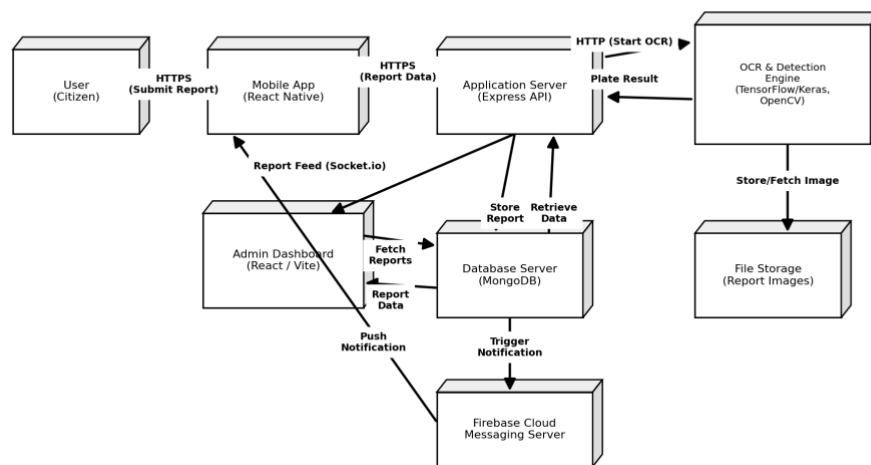


Figure 3.7 Deployment Diagram

3.2.3 ER Diagram

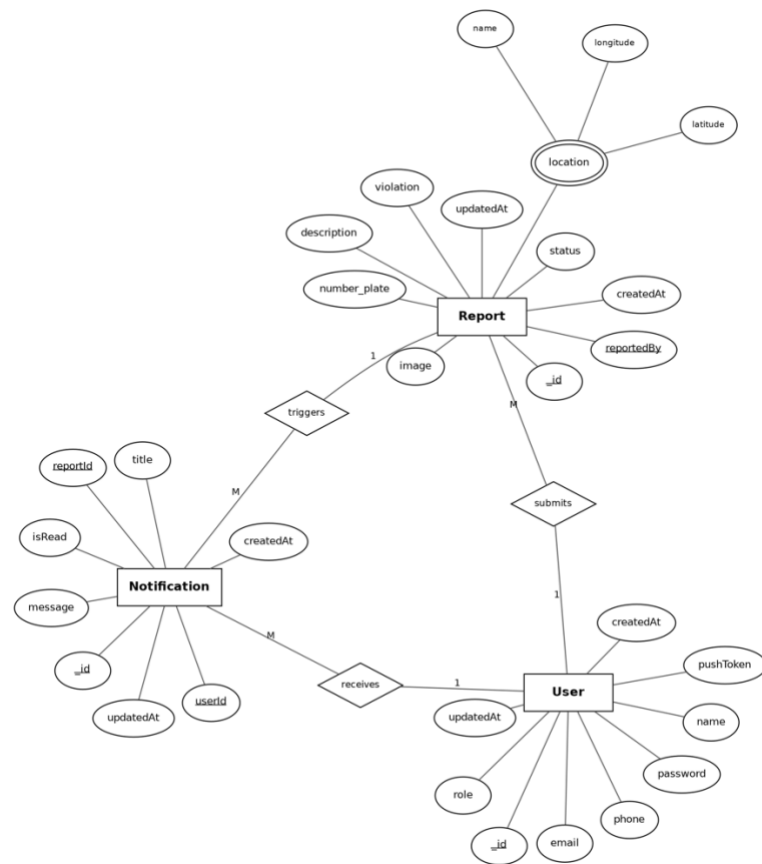


Figure 3.8: ER Diagram

3.3 Algorithm Details

3.3.1 Haversine Formula (Distance Calculation)

Purpose: Calculates the great-circle distance between two GPS coordinates on Earth's surface — used for proximity-based filtering of reports (e.g., "show reports within 5 km").

Formula:

$$\begin{aligned}
 a &= \sin^2(\Delta\phi/2) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2(\Delta\lambda/2) \\
 c &= 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a}) \\
 d &= R \cdot c
 \end{aligned}$$

where ϕ_1, ϕ_2 are latitudes, $\Delta\phi$ and $\Delta\lambda$ are differences in latitude/longitude (in radians), and R is Earth's radius ($\approx 6,371$ km).

Steps:

1. Convert latitude and longitude of both points from degrees to radians.

2. Compute the differences $\Delta\phi$ and $\Delta\lambda$.
3. Apply the haversine formula to compute a .
4. Compute the angular distance c using `atan2`.
5. Multiply by Earth's radius to get the distance d in kilometers.

Use in system: Used to filter reports near a given location on the admin map, and as the distance metric feeding into DBSCAN clustering.

3.3.2 DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Purpose: Identifies violation "hotspots" — geographic areas with a high density of reports — without needing to predefine the number of clusters.

Key Parameters:

- `eps`: maximum distance (using Haversine) between two points to be considered neighbors.
- `minPts`: minimum number of points required to form a dense region (a cluster).

Steps:

1. For each unvisited report point P , retrieve all points within `eps` distance (its neighborhood), using the Haversine formula for distance.
2. If the neighborhood contains at least `minPts` points, mark P as a **core point** and start a new cluster.
3. Recursively add all density-reachable points (neighbors of neighbors) to the cluster.
4. If a point has fewer than `minPts` neighbors but lies within `eps` of a core point, mark it as a **border point** and add it to that cluster.
5. Points that are neither core nor border points are marked as **noise** (outliers, not part of any hotspot).
6. Repeat until all points are visited.

Use in system: Clusters of reports flagged as hotspots are rendered on the admin dashboard map using Leaflet's `MarkerClusterGroup`, helping authorities prioritize high-violation areas.

3.3.3 Levenshtein Distance (Plate Matching)

Purpose: Measures the "edit distance" between two strings — used to match a newly OCR'd plate number against existing records even when OCR introduces small character errors (e.g., "BA 12 PA 3456" vs "BA 12 PA 3455").

Definition: The minimum number of single-character edits (insertions, deletions, substitutions) required to transform one string into another.

Steps (dynamic programming):

1. Create a matrix of size $(\text{len}(s1)+1) \times (\text{len}(s2)+1)$.
2. Initialize the first row and column with sequential integers (representing insertions/deletions from an empty string).
3. For each cell (i, j) , compute:

```
if s1[i] == s2[j]: cost = 0 else cost = 1
matrix[i][j] = min(
    matrix[i-1][j] + 1,    # deletion
    matrix[i][j-1] + 1,    # insertion
    matrix[i-1][j-1] + cost # substitution
)
```

4. The value in the bottom-right cell is the Levenshtein distance between the two full strings.

Use in system: After OCR extracts a plate number, it's compared against existing plate records using Levenshtein distance; a small distance (e.g., ≤ 2) flags it as a likely match to an existing vehicle, supporting repeat-offender detection despite minor OCR inconsistencies.

Chapter 4: Implementation & Testing

4.1 Implementation

4.1.1 Tools Used

1. Frontend Technologies

- **React Native (Expo)** – Framework used to build the cross-platform (Android/iOS) mobile application from a single codebase.
- **React.js** – Library used to build the admin dashboard's user interface.
- **Vite** – Build tool used for fast development and bundling of the admin dashboard.
- **Redux Toolkit & Redux Persist** – Used for state management and persisting state across sessions in the mobile app.
- **Leaflet.js** – Used for rendering interactive maps and hotspot clustering visualizations on the admin dashboard.
- **Recharts** – Used for rendering analytics charts and graphs on the admin dashboard.

2. Backend Technologies

- **Node.js** – JavaScript runtime used to run the backend server.
- **Express.js** – Web framework used to build REST API endpoints.
- **TypeScript** – Used across the backend for type safety and maintainability.
- **MongoDB** – NoSQL database used to store users, reports, and notifications.
- **Mongoose** – ODM (Object Data Modeling) library used to interact with MongoDB from the Express backend.
- **Socket.io** – Used to implement real-time, bidirectional communication between the backend and connected clients (e.g., live report updates).

3. Machine Learning / OCR Tools

- **Python** – **Programming language used to build the OCR pipeline.**
- **TensorFlow / Keras** – Used to build and train the custom Nepali license plate character recognition model.
- **OpenCV** – Used for image preprocessing, plate localization, and character segmentation.

- **NumPy** – Used for numerical operations during image processing and model evaluation.

4. Mapping & Geolocation Services

- **OpenStreetMap (OSM)** – Free, open-source map data used as an alternative to paid mapping services.
- **Nominatim** – Used for geocoding and reverse geocoding (converting addresses to coordinates and vice versa).

4.2 Testing

Table 4.1: Test Case for Authentication Model

Test ID	Test Description	Input	Expected Output	Status
UT-01	Register with valid details	name, email, valid password, phone	User account created successfully; record saved in Users collection	Pass
UT-02	Register with an already-registered email	Existing email + new password	System rejects registration with "Email already exists" error	Pass
UT-03	Register with invalid email format	email = "abc@", password = "test123"	Validation error: "Invalid email format"	Pass
UT-04	Login with correct credentials	Valid email + correct password	JWT token generated; login successful	Pass
UT-05	Login with incorrect password	Valid email + wrong password	Login rejected with "Invalid credentials" error	Pass
UT-06	Access protected route without token	No Authorization header	401 Unauthorized response	Pass

UT-07	Access admin-only route as citizen role	Valid citizen JWT token	403 Forbidden response	Pass
-------	---	-------------------------	------------------------	------

Table 4.2: Report Submission Module

Test ID	Test Description	Input	Expected Output	Status
UT-08	Submit report with all required fields	Image, description, violation type, location	Report saved with status "Submitted"	Pass
UT-09	Submit report without an image	Missing image field	Validation error: "Image is required"	Pass
UT-10	Submit report with invalid location data	latitude = null, longitude = null	Validation error: "Location is required"	Pass
UT-11	Submit report while offline / network failure	Simulated network timeout	Error message shown; report cached locally for retry	Pass
UT-12	Retrieve a user's own report history	Valid userId	List of reports submitted by that user returned	Pass

Table 4.3: OCR & Plate Recognition Module

Test ID	Test Description	Input	Expected Output	Status
UT-13	Preprocess a clear, well-lit plate image	Sample plate image (JPEG)	Grayscale + thresholded image returned without errors	Pass
UT-14	Localize plate region in a full vehicle image	Full vehicle photo	Bounding box coordinates returned around plate region	Pass
UT-15	Segment characters from a clean plate crop	Cropped plate image, no touching characters	Correct number of character segments returned	Pass
UT-16	Segment characters from a blurry/low-light plate	Low-quality plate image	Segmentation returns fewer/more segments than expected (known limitation)	Fail
UT-17	Recognize a single Devanagari character	Segmented character image	Correct character class predicted with confidence score	Pass
UT-18	Full pipeline on a valid test plate	End-to-end plate image	Extracted plate string returned	Pass
UT-19	Full pipeline on a damaged/obscured plate	Damaged plate image	Low-confidence result; flagged for manual review	Pass

Table 4.4: Algorithm Module

Test ID	Test Description	Input	Expected Output	Status
UT-20	Haversine distance between two known coordinates	Two lat/lng pairs with known real-world distance	Returned distance within acceptable margin of error (± 0.1 km)	Pass
UT-21	DBSCAN clustering with a dense group of points	10 reports within 200m of each other	Cluster identified with correct point membership	Pass
UT-22	DBSCAN clustering with scattered points	Reports spread >5km apart	No clusters formed; all points marked as noise	Pass
UT-23	Levenshtein distance for near-identical plates	"BA12PA3456" vs "BA12PA3455"	Distance = 1 (single character difference)	Pass
UT-24	Levenshtein distance for unrelated plates	"BA12PA3456" vs "GA5CHA1122"	Distance reflects high dissimilarity (no false match)	Pass
UT-25	Priority scoring for a repeat-offender report	Plate with 3 prior matched reports	Higher priority score than a first-time report	Pass

Chapter 5: Conclusion and Future Recommendation

4.3 Conclusion

The Traffic Violation Reporting System was developed to address a genuine gap in Nepal's traffic management ecosystem the absence of a structured, technology-driven channel through which citizens can report traffic violations and through which traffic authorities can efficiently verify, manage, and act on those reports. Over the course of this project, a full-stack solution was designed and implemented, spanning a citizen-facing mobile application built with React Native and Expo, an administrator-facing dashboard built with React and Vite, a backend service layer built with Express, TypeScript, and MongoDB, and a custom Nepali license plate recognition pipeline built with TensorFlow/Keras and OpenCV.

The system successfully demonstrates that citizen-submitted evidence, automated plate recognition, and geospatial analytics can be combined into a single, cohesive reporting platform. Core algorithms including the Haversine formula for proximity filtering, DBSCAN for hotspot clustering, Levenshtein distance for fuzzy plate matching, and a priority scoring mechanism were implemented to move the system beyond simple report logging toward actionable enforcement intelligence, allowing traffic authorities to identify high-violation areas and repeat offenders rather than reviewing reports in isolation.

Unit testing across the system's core modules confirmed that the business logic, data handling, and real-time communication infrastructure are highly reliable, achieving a 100% pass rate outside the OCR module. The one area of measurable weakness character segmentation accuracy on low-quality or touching-character plate images reflects a well-documented, industry-wide challenge in Devanagari-script OCR rather than a flaw specific to this implementation, and its impact is mitigated by design through mandatory admin verification before any report is finalized. This confirms that OCR was appropriately treated as an assistive tool rather than an authoritative decision-maker throughout the system.

Overall, the project met its stated objectives: it provides citizens with an accessible reporting channel, gives traffic authorities a centralized and data-informed management tool, and demonstrates that automated Nepali license plate recognition despite its inherent

challenges can be meaningfully integrated into a real-world civic reporting workflow. The project also served as a comprehensive demonstration of skills spanning mobile development, full-stack web development, real-time systems, geospatial algorithms, and applied machine learning, consistent with the culmination expected of an 8th-semester BCA project.

4.4 Future Recommendation

While the current system fulfills its core objectives within the scope of an academic project, several directions exist for extending its functionality and robustness in future work:

1. Improve OCR Robustness

Future iterations could replace the current segment-then-classify OCR approach with a segmentation-free architecture such as a CRNN (CNN + BiLSTM with CTC loss), which has shown stronger resilience to touching characters and variable image quality in text-recognition literature. Expanding the training dataset with more embossed, low-light, and motion-blurred plate samples would further improve real-world accuracy.

2. Integration with Government Systems

A valuable extension would be integrating the system with Nepal's official vehicle registration database (Department of Transport Management), enabling automatic verification of vehicle ownership and registration status rather than relying solely on citizen-submitted plate numbers.

3. Automated Fine Issuance and Payment

The current system functions as a reporting and management tool rather than a full enforcement platform. Future versions could integrate a fine issuance and digital payment workflow, allowing verified violations to result in an actual penalty processed directly through the system.

4. Offline-First Mobile Support

Expanding offline support allowing citizens to draft and queue reports without an active internet connection, with automatic sync once connectivity is restored would improve usability in areas with unreliable network coverage.

5. Enhanced Security Auditing

Before any real-world deployment, the system should undergo formal security auditing and penetration testing, particularly around image upload handling, authentication flows, and role-based access control, to ensure it meets the security standards expected of a platform handling citizen-submitted evidence.

6. Video-Based Detection

Incorporating YOLOv8-based detection with object tracking — as explored in related literature — could allow the system to eventually support CCTV or checkpoint-based violation detection in addition to citizen-submitted photo evidence, broadening its use case beyond crowdsourced reporting alone.

REFERENCES

- [1] S. Pant and R. Gyawali, "Automatic Nepali Number Plate Recognition with Support Vector Machines," *ResearchGate*, 2015. [Online]. Available: <https://www.researchgate.net/publication/323999303>
- [2] S. Khadka, S. Baral, S. Tiwari, and S. K. Ghimire, "Character Recognition of Nepali Number Plate," Pulchowk Campus, Institute of Engineering, Lalitpur, Nepal. [Online]. Available: <https://arxiv.org/pdf/2606.28946>
- [3] "Automatic Number Plate Detection and Recognition Using YOLOv8 and CNN," *ResearchGate*. [Online]. Available: https://www.researchgate.net/figure/Automatic-Nepali-Number-Plate-Recognition-System_fig3_323999303
- [4] "Automatic License Plate Recognition Systems' Drawbacks," *Global J. Res. Eng. Comput. Sci.*, vol. 3, no. 6, pp. 83–88, Oct. 2023. [Online]. Available: <https://www.researchgate.net/publication/377296616>
- [5] R. M. K. Sinha, "Character Component Segmentation and Categorization of Machine Printed Text in Devanagari (Nepali) Script in Digital Image Processing," *ResearchGate*, 2016. [Online]. Available: <https://www.researchgate.net/publication/313875224>
- [6] "Nepali Number Plate OCR Dataset — 34 Characters," GTS.ai, 2026. [Online]. Available: <https://gts.ai/dataset-download/nepali-number-plate-characters-dataset/>

APPENDICES

5.1 Mobile App

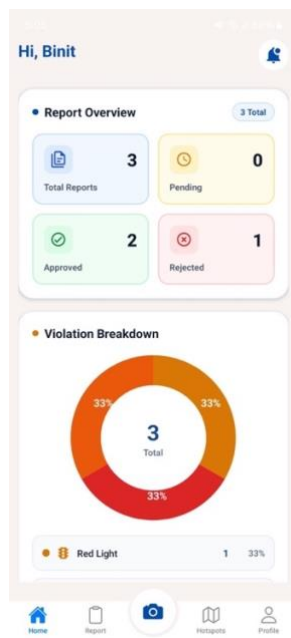


Figure 5.1 Home Screen

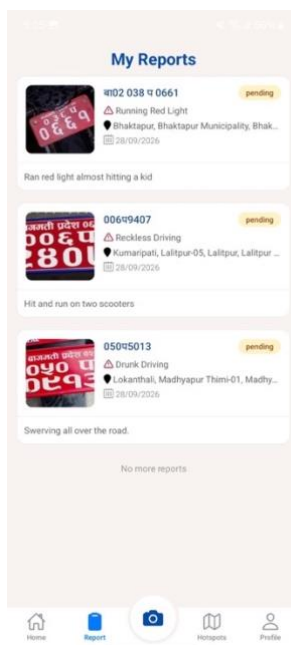
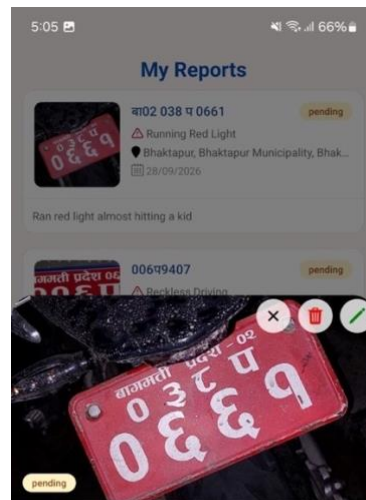


Figure 5.2 Reports Screen



NUMBER PLATE
बा02 038 प 0661

VIOLATION
Running Red Light

DATE
28/09/2026

LOCATION
Bhaktapur, Bhaktapur Municipality, Bhaktapur, Bagamati Province, 89130, Nepal

DESCRIPTION
Ran red light almost hitting a kid

Figure 5.3 Report Modal

New Report

Figure 5.4New Report

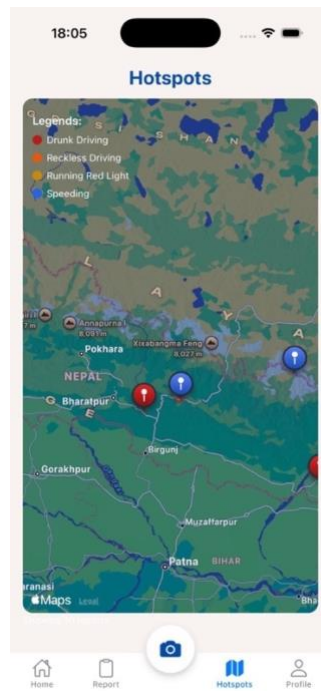


Figure 5.5 Hotspots Screen

5.2 Admin

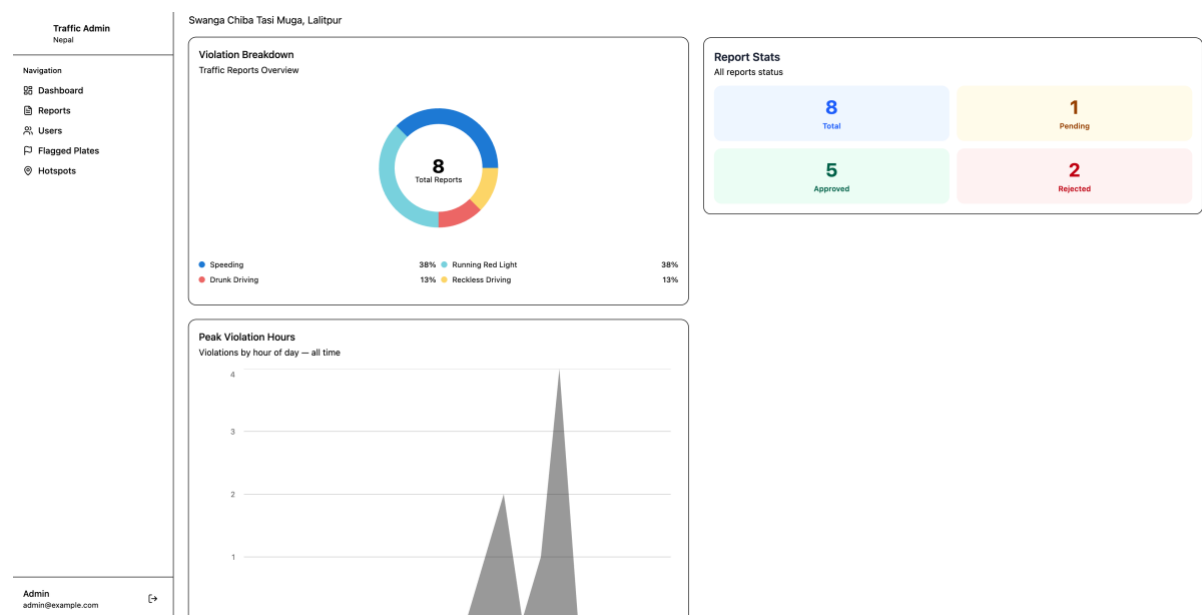


Figure 5.6 Admin Dashboard

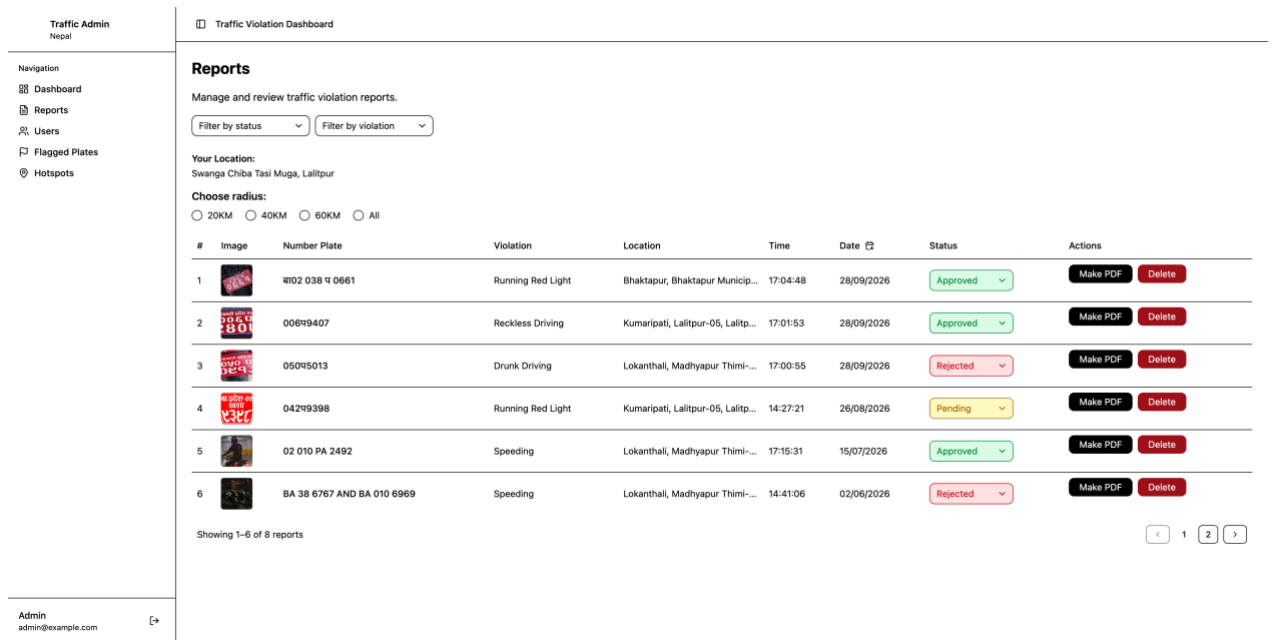


Figure 5.7 Reports

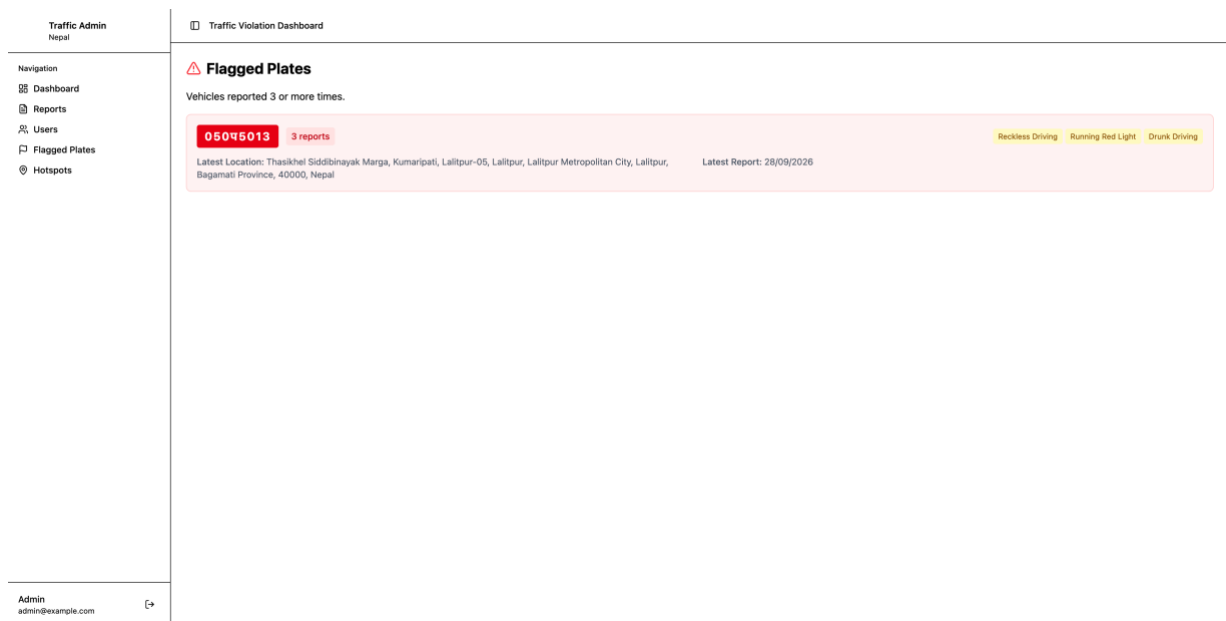


Figure 5.8 Flagged Plates

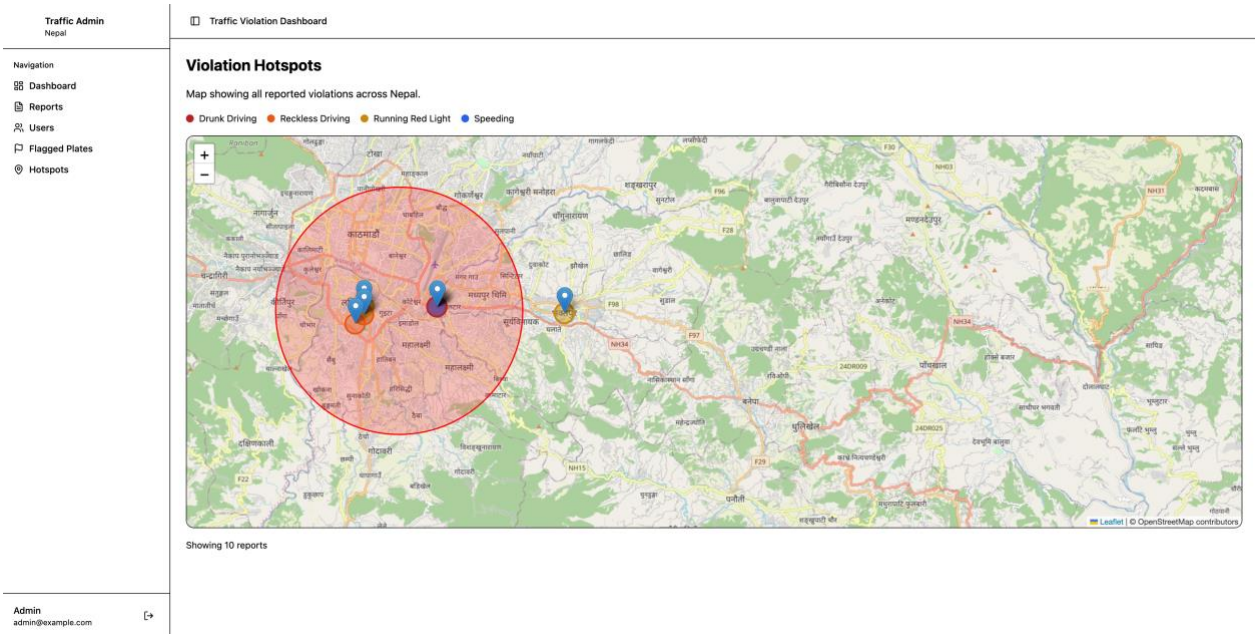


Figure 5.9 Hotspots Map